



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA INGENIERÍA EN SOFTWARE

TEMA:
INSTALACIÓN DE LOS ENTORNOS Y PRIMEROS
SCRIPT PHP Y PERL

PRESENTADO POR:
TAIPE MORA ZAIDA MELISSA

FECHA:
VIERNES, 05 DE JUNIO DEL 2026
(Semana 5)

1. Instalación del entorno desarrollado (XAMP)

Se llevó a cabo la instalación y configuración inicial del entorno de desarrollo local utilizando la distribución XAMPP. Tras instalar los componentes esenciales (Apache, PHP y Perl), se procedió a inicializar con éxito los servicios de Apache y MySQL desde el panel de control. Finalmente, se realizó una auditoría de configuración en el archivo `httpd.conf` de Apache, asegurando que el módulo encargado de procesar scripts CGI (`mod_cgi.so`) se encuentre activo y que la directiva `ScriptAlias` apunte correctamente al directorio destinado para la ejecución de los archivos de Perl (`cgi-bin`).

```
XAMPP for Windows

Setting environment for using XAMPP for Windows.
1EASU1741@ZAIDA c:\xampp
# php -v
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
Copyright (c) The PHP Group
Zend Engine v4.2.12, Copyright (c) Zend Technologies
```

Figura 1. Verificación de la versión PHP (salida de `php -v`)

```
1EASU1741@ZAIDA c:\xampp
# perl -v

This is perl 5, version 32, subversion 1 (v5.32.1) built for MSWin32-x64-multi-thread
Copyright 1987-2021, Larry Wall

Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl". If you have access to the
Internet, point your browser at http://www.perl.org/, the Perl Home Page.
```

Figura 2. Verificación del entorno de ejecución Perl (salida de `perl -v`)

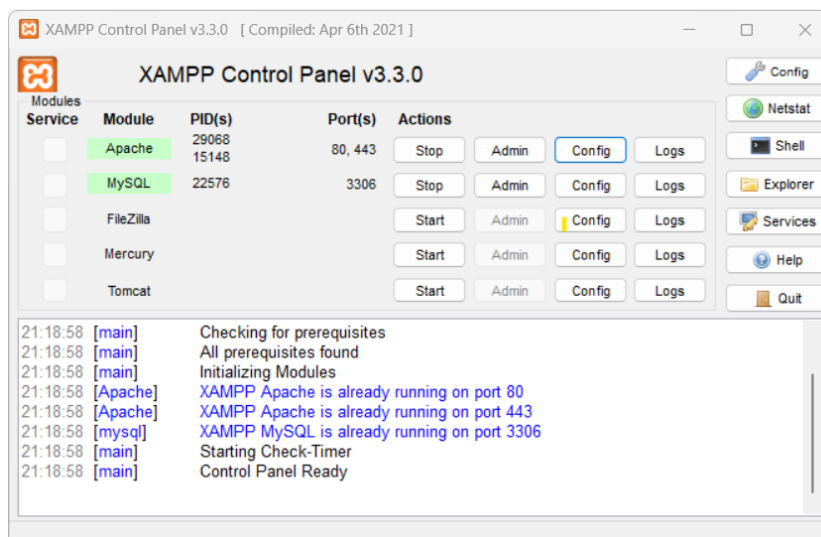


Figura 3. Panel de control de XAMPP con los servicios de Apache y MySQL activos.

2. Arquitectura de Archivos e Interfaz de Usuario HTML

Con el propósito de garantizar un análisis comparativo y simétrico, se diseñó una interfaz unificada centralizada en el archivo `formulario.html`, ubicado en el directorio raíz del proyecto. Este documento hace uso del atributo estándar `formaction` en sus respectivos

botones de acción, lo que permite bifurcar y redirigir el flujo de datos (mediante el método POST) hacia los distintos entornos de procesamiento independientes que configuran la arquitectura local.

En esta sección se exhibe la estructura visual del formulario de captura de datos tal como es interpretado por el navegador web.

← → ↻ ⓘ localhost/practica5/formulario.html ☆ Z En pausa

Formulario de registro

Completa todos los campos. Prueba con PHP o con PERL/CGI.

Nombre completo *

Ej: Maria Garcia

Correo electronico *

usuario@dominio.com

Edad (1-120) *

Ej: 22

Mensaje *

Escribe tu mensaje aquí...

Enviar con PHP Enviar con PERL/CGI

Practica 5 - Aplicaciones Web - UTEQ 2025-2026

Figura 4. Vista de la interfaz del formulario unificado en el navegador del cliente.

3. Validación del Sistema y Casos de Prueba

Con el objetivo de verificar la resiliencia estructural y la lógica de validación de ambos *backends* (PHP y Perl), se ejecutaron 10 escenarios de control operacional basados en los requerimientos metodológicos establecidos.

3.1 Caso 1: Gestión de Campos Vacíos

Envío del formulario sin contener información con el fin de evaluar la capacidad del servidor para capturar globalmente los errores por omisión de datos.

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

- Nombre: El nombre no puede estar vacío.
- Email: El correo electrónico no tiene un formato válido.
- Edad: La edad debe ser un número entero entre 1 y 120.
- Mensaje: El mensaje no puede estar vacío.

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

PERL CGI Resultado del procesamiento

Se encontraron errores:

- Edad: La edad debe ser un número entero entre 1 y 120.
- Email: El correo electrónico no tiene formato válido.
- Mensaje: El mensaje no puede estar vacío.
- Nombre: El nombre no puede estar vacío.

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

3.2 Caso 2: Sintaxis de Correo Electrónico (Ausencia de Dominio)

Evaluación del filtro restrictivo de formato mediante la inserción de una cadena incompleta (usuario@).

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Email: El correo electrónico no tiene un formato válido.

[-- Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Email: El correo electrónico no tiene formato válido.

[-- Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

3.3 Caso 3: Sintaxis de Correo Electrónico (Ausencia de Arroba)

Validación de la máscara de entrada para correos electrónicos utilizando la estructura inválida usuariodominio.com.

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Email: El correo electrónico no tiene un formato válido.

[-- Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Email: El correo electrónico no tiene formato válido.

[-- Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

3.4 Caso 4: Restricción de Rango (Edad Negativa)

Prueba perimetral de límites numéricos para verificar el comportamiento del sistema al recibir un valor entero negativo (-20).

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[-- Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[-- Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

3.5 Caso 5: Restricción de Rango (Edad Excesiva)

Prueba de límite superior en el parámetro de edad, enviando un valor síncronamente inválido que supera el umbral lógico permitido (200).

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[-- Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[-- Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

3.6 Caso 6: Validación de Tipo de Dato (Edad No Numérica)

Inyección de caracteres alfabéticos o cadenas de texto (e) en el campo diseñado exclusivamente para recibir valores enteros.

PHP 8.2 Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[-- Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

PERL/CGI Resultado del procesamiento

Se encontraron errores:

Edad: La edad debe ser un número entero entre 1 y 120.

[-- Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Método: POST

3.7 Caso 7: Mitigación de Cross-Site Scripting (XSS) en Campo de Nombre

Inyección de la etiqueta estructurada `<script>alert('xss')</script>`. En este escenario se audita el saneamiento de datos mediante la conversión a entidades HTML, verificando su comportamiento tanto en la interfaz del cliente como en el código fuente renderizado por el servidor.

- **Evidencias en PHP:** *Captura de pantalla del resultado y fragmento del código fuente.*

PHP 8.2

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	<script>alert(1)</script>
Email	ana@hotmail.com
Edad	20 años
Mensaje	Holaa

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ::1

[illegible]

- **Evidencias en Perl:** *Captura de pantalla del resultado y fragmento del código fuente.*

A screenshot of a web browser displaying a message in Spanish: "Datos recibidos y validados correctamente." (Data received and validated correctly). Below the message is a green button labeled "Resumen de los datos:" (Summary of the data:). Underneath the button is a table with two columns: "Campo" (Field) and "Valor recibido" (Received value). The table contains four rows of data: "Nombre" (Name) with the value "<script>alert(1)</script>", "Email" with "ana@hotmail.com", "Edad" (Age) with "20 anos", and "Mensaje" (Message) with "Holaa". At the bottom left of the browser window, there is a blue link that says "+ Volver" (Back).

[illegible]

3.8 Caso 8: Mitigación de XSS Activo en Campo de Mensaje

Inyección del elemento reactivo e interactivo `<img src=x onerror=alert(1)>`. Se valida el correcto enmascaramiento y escape de caracteres especiales en ambos intérpretes.

- **Evidencias en PHP:** *Captura de pantalla del resultado y fragmento del código fuente.*

PHP 8.2

Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Ana
Email	ana@hotmail.com
Edad	20 años
Mensaje	

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Método: POST | IP: ...1

```

1 <?php
2 <html type="html"
3 <html lang="es">
4 <head>
5 <meta charset="UTF-8">
6 <title>Resultado PHP</title>
7 <style>
8 body { font-family: Arial, sans-serif; max-width: 600px; margin: 2rem auto; padding: 0 1rem; }
9 .error { color: #c0392b; background: #fde5e5; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
10 .info { color: #1abc9c; background: #e6f2ff; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #1abc9c; margin: 4px 0; }
11 table { width: 100%; border-collapse: collapse; }
12 td, th { padding: 8px 12px; border: 1px solid #ddd; text-align: left; }
13 th { background: #7f7b3b; color: white; }
14 tr:nth-child(even) { background: #f9f9f9; }
15 back { display: inline-block; padding: 2px 10px; border-radius: 12px; font-size: 0.8rem; background: #7f7b3b; color: white; }
16 </style>
17 </head>
18 <body>
19 <div class="badge">PHP 8.2</div> Resultado del procesamiento</h1>
20
21 <div class="col"><strong>Datos recibidos y validados correctamente.</strong></div>
22 <h2>Resumen de los datos:</h2>
23 <table>
24 <tr><th>Campo</th><th>Valor recibido</th></tr>
25 <tr><td>Nombre</td><td>Ana</td></tr>
26 <tr><td>Email</td><td>ana@hotmail.com</td></tr>
27 <tr><td>Edad</td><td>20 años</td></tr>
28 <tr><td>Mensaje</td><td>¡&img src=x onerror=alert(1)!&img</td></tr>
29 </table>
30 <a href="formulario.html">¡&arr; Volver al formulario</a></p>
31 <p style="font-size:0.8rem; color:#888;">
32 Procesado por PHP 8.2.12 | Metodo: POST | IP: :11 </p>
33 </body>
34 </html>

```

- Evidencias en Perl: *Captura de pantalla del resultado y fragmento del código fuente.*
-

PERL/CGI Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Ana
Email	ana@hotmail.com
Edad	20 años
Mensaje	

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4 <meta charset="UTF-8">
5 <title>Resultado PERL/CGI</title>
6 <style>
7 body { font-family: Arial, sans-serif; max-width: 600px; margin: 2rem auto; padding: 0 1rem; }
8 .error { color: #c0392b; background: #fdecea; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #c0392b; margin: 4px 0; }
9 .exit { color: #1e6b3c; background: #eaf5ee; padding: 8px 12px; border-radius: 4px; border-left: 4px solid #1e6b3c; }
10 table { width:100%; border-collapse: collapse; }
11 td, th { padding:8px 12px; border:1px solid #ddd; }
12 th { background:#3d8b3d; color:white; }
13 tr:nth-child(even) { background:#f5f5f5; }
14 .badge { display:inline-block; padding:2px 10px; border-radius:12px; font-size:0.8rem; background:#3d8b3d; color:white; }
15 </style>
16 </head>
17 <body>
18 <div class="error">PERL/CGI</div> Resultado del procesamiento</div>
19 <div class="exit"><strong>Datos recibidos y validados correctamente.</strong></div>
20 <h2>Resumen de los datos:</h2>
21 <table>
22 <tr><th>Campos</th><th>Valor recibido</th></tr>
23 <tr><td>Nombre</td><td>Ana</td></tr>
24 <tr><td>Email</td><td>ana@hotmail.com</td></tr>
25 <tr><td>Edad</td><td>20 años</td></tr>
26 <tr><td>Mensaje</td><td><img src=x onerror=alert(1)&gt;</td></tr>
27 </table>
28 <p>a href="#practica5/formulario.html"&larr; Volver</a></p>
29 <div>
30 <p style="font-size:0.8rem; color:#888;">Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST</p>
31 </div>
32 </html>

```

3.9 Caso 9: Flujo Normal con Datos Válidos

Procesamiento óptimo del formulario utilizando parámetros que cumplen estrictamente con todas las reglas de negocio y especificaciones del sistema.

PHP 8.2 Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria
Email	maria@hotmail.com
Edad	22 años
Mensaje	Este es mensaje de prueba

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

PERL/CGI Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria
Email	maria@hotmail.com
Edad	22 años
Mensaje	Este es mensaje de prueba

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

3.10 Caso 10: Soporte de Caracteres Especiales y Graffías Castellanas en Nombre

Envío de nombres compuestos que incluyen tildes, espacios y guiones (María José Muñoz-López) para constatar la correcta codificación de caracteres en la respuesta del servidor.

PHP 8.2 Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria Jose Munoz-Lopez
Email	maria@hotmail.com
Edad	22 años
Mensaje	Holaa

[← Volver al formulario](#)

Procesado por PHP 8.2.12 | Metodo: POST | IP: ::1

PERL/CGI Resultado del procesamiento

Datos recibidos y validados correctamente.

Resumen de los datos:

Campo	Valor recibido
Nombre	Maria Jose Munoz-Lopez
Email	maria@hotmail.com
Edad	22 años
Mensaje	Holaa

[← Volver](#)

Procesado por Perl 5.32.1 con CGI.pm | Metodo: POST

4. Análisis Comparativo: PHP 8.2 frente a Perl 5.38 (Entorno CGI)

La tabla que se presenta a continuación sintetiza los contrastes operativos e instrumentales detectados durante el desarrollo de la práctica entre **PHP 8.2** y **Perl 5.38** (empleando el **módulo CGI.pm**). Se incorpora un apartado de valoración diagnóstica derivado de la interacción directa con ambos entornos de ejecución.

Criterio	PHP 8.2	Perl 5.38 con CGI.pm	Valoración Diagnóstica
Integración con Apache	Ejecución nativa mediante mod_php. El intérprete reside de forma persistente dentro del hilo del servidor, optimizando la memoria al omitir la bifurcación de subprocesos.	Mecanismo CGI estándar. El servidor web debe instanciar un proceso independiente de Perl por cada solicitud entrante, incrementando la carga operativa (<i>overhead</i>).	La arquitectura de PHP ofrece una experiencia notablemente más fluida al mitigar el costo computacional de la inicialización de procesos concurrentes.
Lectura de Parámetros	Captura directa mediante variables superglobales o abstracciones nativas como filter_input(INPUT_POST, ...).	Extracción basada en programación orientada a objetos a través del método \$cgi->param('campo').	El enfoque de PHP resulta más pragmático e inmediato, mientras que Perl demanda una sintaxis más verbosa al requerir la inicialización explícita del objeto CGI.
Mitigación de XSS	Implementación de htmlspecialchars() parametrizada bajo la directiva ENT_QUOTES.	Utilización del método estático del módulo: CGI::escapeHTML().	Ambos entornos logran el mismo nivel de seguridad. La alternativa de Perl es sintácticamente más compacta, aunque menos intuitiva a nivel comercial.
Validación de Email	Abstracción nativa integrada en el núcleo del lenguaje (FILTER_VALIDATE_EMAIL).	Evaluación estructural construida mediante expresiones regulares específicas: / [^] .+@.+\.[a-zA-Z]{2,}\$/.	PHP proporciona una ventaja competitiva al estandarizar el filtro. En Perl, la construcción manual del patrón añade complejidad estructural al código.
Validación de Enteros	Uso de constantes nativas (FILTER_VALIDATE_INT) con soporte nativo para acotamiento de rangos.	Combinación de lógica condicional y expresiones regulares de coincidencia de dígitos (/^d+/).	PHP agiliza la implementación mediante funciones nativas, en contraste con el enfoque modular de Perl que incrementa el número de líneas requeridas.
Cabeceras HTTP	Emisión automatizada por el intérprete; el método header() se reserva exclusivamente para flujos de redirección.	Gestión obligatoria en el flujo de salida inicial a través de \$cgi->header() previo al renderizado de datos.	La rigidez de Perl requiere precaución; omitir la cabecera provoca fallos críticos del servidor (Error 500) sin arrojar trazas descriptivas en primera instancia.
Modo Estricto	Activación opcional mediante la directiva de tipado declare(strict_types=1);.	Directivas restrictivas de compilación explícitas: use strict; y use warnings;.	Ambos paradigmas cumplen con el mismo propósito preventivo, obligando a mantener buenas prácticas de tipado y declaración de variables.
Renderizado HTML	Inserción directa o embebida intercalando bloques funcionales con sintaxis HTML estándar.	Emisión de cadenas a través de instrucciones print, estructuras <i>Heredoc</i> o el método \$cgi->start_html().	El formato <i>Heredoc</i> en Perl agiliza la gestión de plantillas extensas, a pesar de que su curva de legibilidad inicial resulta demandante.
Permisos del Sistema	Gestión transparente y desatendida bajo arquitecturas Windows con XAMPP.	Requisito explícito de configuración de permisos (chmod 755) en entornos basados en UNIX dentro del directorio cgi-bin.	Para despliegues en servidores Linux, Perl impone pasos de configuración adicionales a nivel de sistema de archivos que pueden inducir a errores operacionales.
Curva de Aprendizaje	Moderada. Sintaxis estructurada de fácil adopción y con una amplia documentación comunitaria.	Elevada. Sintaxis altamente expresiva y polimórfica (paradigma <i>TIMTOWTDI</i>), demandando mayor abstracción.	PHP facilita una curva de adopción acelerada desde cero, mientras que Perl exige un periodo de familiarización técnica más prolongado.
Ecosistema Actual	Aplicaciones web dinámicas de alta escala, Sistemas de Gestión de Contenidos (CMS) y APIs RESTful.	Scripting del sistema, administración de servidores, bioinformática y procesamiento masivo de cadenas de texto.	PHP se posiciona como el estándar para el desarrollo web moderno, relegando a Perl hacia nichos especializados de automatización y analítica de datos.

5. Reflexión Final

El desarrollo de esta práctica permitió contrastar de manera directa dos soluciones tecnológicas de servidor fundamentadas sobre filosofías de diseño diametralmente opuestas: PHP 8.2 y Perl 5.38 con el soporte de CGI.pm. El punto de divergencia más crítico radica en los mecanismos de seguridad aplicados a la sanitización e integridad de los datos de entrada. Mientras que PHP provee una arquitectura nativa simplificada mediante filtros integrados (`filter_input`), consolidando un flujo intuitivo que minimiza el volumen de código, Perl delega esta responsabilidad directamente en la pericia del desarrollador a través de expresiones regulares manuales; una aproximación con alta flexibilidad, pero propensa a vulnerabilidades lógicas ante errores de diseño en el patrón.

Adicionalmente, el rendimiento asociado a la comunicación con el servidor web expone un contraste arquitectónico sustancial. El acoplamiento de PHP como módulo nativo dentro de Apache optimiza la concurrencia al no generar procesos satélites. En contraposición, el modelo clásico de CGI en Perl introduce una penalización en el rendimiento del servidor ante cargas masivas de usuarios debido a la continua ramificación (*forking*) de procesos independientes para dar respuesta a cada petición HTTP.

En conclusión, la selección tecnológica para un despliegue real queda sujeta al contexto del proyecto: se optará por PHP en el diseño de plataformas web dinámicas contemporáneas debido a su mantenibilidad, rápida adopción y extensa documentación técnica. Por otra parte, Perl se consolida como la herramienta de elección en escenarios orientados al procesamiento por lotes (*batch processing*), automatización de tareas en sistemas operativos o entornos científicos como la bioinformática, áreas donde su potencia sintáctica y manipulación de texto resultan óptimas.